KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
Deemed to be University U/S 3 of the UGC Act, 1956

Land–Water Boundary Detection

Using a Custom CNN Built Entirely From Scratch

Final Year Project Report

Computer Vision · Image Processing · CNN Fundamentals

Project Domain	Satellite Image Analysis & Remote Sensing
Core Technology	Custom CNN (No ML Frameworks) + Flask Web App
Language	Python 3.10.0
Libraries	NumPy · OpenCV · Flask · Matplotlib
Evaluation	Custom Boundary Coverage Score (Intersection / Predicted)
Result	Avg. Score: 0.65 across 10 Diverse Satellite Images

ANKITT KUSHARY
23053027
CSE - 48

Academic Year 2025 – 2026

Department of Computer Science & Engineering

1. Introduction

Background & Motivation

Automatic detection of land–water boundaries from satellite imagery is a critical task in remote sensing — underpinning flood mapping, shoreline erosion tracking, and disaster response. Conventional solutions rely either on costly manual annotation or on large pre-trained deep learning models requiring extensive labelled datasets and GPU resources. This project challenges both approaches by delivering a fully deterministic, interpretable pipeline with zero training.

Problem Statement

Given a raw satellite image depicting any water body — ocean coastline, river, lake, or turbid/algae-laden water — the system must:

- Accurately detect the spatial land–water boundary across diverse terrain types.
- Overlay the detected boundary as a red contour on the original colour image.
- Operate without any model training, backpropagation, or pretrained weights.
- Expose results through an interactive Flask web interface in real time.

Academic Constraints & Significance

The project was executed under a strict requirement: **no deep learning framework** (TensorFlow, PyTorch, Keras) may be used for core processing. Every convolution, activation, pooling, and scoring operation is coded explicitly in Python using only NumPy and OpenCV. This forces a deep engagement with CNN internals — padding, kernel correlation, ReLU, and statistical feature aggregation — producing a fully transparent, auditable system where every numerical step can be analytically traced.

Key Highlights at a Glance

Custom CNN from scratch No backpropagation or training	**Deterministic filter bank** 4 fixed kernels: Sobel-X/Y, Laplacian, Diagonal
Patch-based texture analysis 16×16 sliding window, stride 4	**Composite water scoring** Smoothness + Blue dominance − Gradient energy
Flask web deployment Real-time upload & boundary viz	**Custom evaluation metric** Boundary coverage score (avg. 0.64 / 10 images)

2. Model Architecture

The system is a six-stage deterministic pipeline — entirely free of trainable parameters. Each stage is encapsulated in its own Python module for clean separation of concerns.

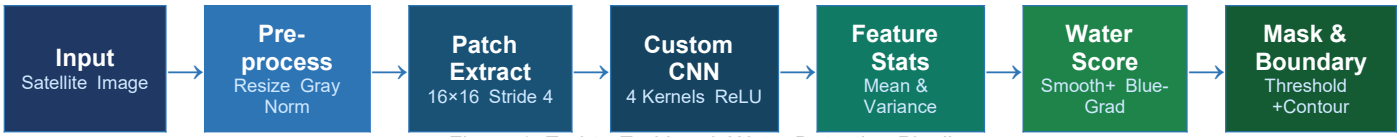

Figure 1: End-to-End Land–Water Detection Pipeline

Stage 1 — Preprocessing (utils/preprocess.py)

Raw satellite image is loaded, converted to grayscale, resized to 256 × 256, and normalised to [0, 1] float range. Standardised dimensions ensure consistent downstream patch enumeration.

Stage 2 — Patch-Based Segmentation (segmentation/patch_segmenter.py)

A 16 × 16 sliding window with stride 4 densely covers the image. For each patch, both grayscale and colour sub-images are extracted so texture and colour cues can be jointly analysed.

Stage 3 — Custom CNN Feature Extraction (cnn/)

Four deterministic 3 × 3 kernels form the filter bank: Sobel-X (horizontal edges), Sobel-Y (vertical edges), Laplacian (blobs/corners), and a Diagonal contrast filter. Convolution is hand-coded as nested-loop zero-padded correlation. Each output passes through ReLU activation; absolute values are retained as feature maps.

Stage 4 — Statistical Features (features/statistics.py)

Per-map mean and variance are averaged across all four feature maps into two scalars. Low variance → smooth texture → water-like region. High variance → chaotic, edge-rich content → land region.

Stage 5 — Water Scoring & Mask

Composite score per patch: water_score = 0.6 × smoothness + 0.3 × blue_score − 0.5 × gradient_energy. Scores accumulate into a score map, normalised to [0,1] and binarised at 0.5. Largest connected component is retained; 7 × 7 morphological close/open refines the mask.

Stage 6 — Boundary Extraction (main.py)

OpenCV findContours extracts polygon contours from the binary water mask. Contours are drawn in red (BGR 0,0,255) with 2 px width on the resized colour original, producing the final annotated output.

3. Sample Code — All 13 Pipeline Steps

Step 1 Raw Input Image [utils/preprocess.py]

Load the satellite image from disk using OpenCV. The raw BGR array and its channel histograms show the original pixel distribution before any processing.

```
# utils/preprocess.py
import cv2

def load_image(path):
    image = cv2.imread(path)    # returns HxWx3 BGR numpy array
    return image

raw_bgr = load_image('data/testcases/1.png')
# Shape: (H, W, 3)  |  dtype: uint8  |  range: [0, 255]
```

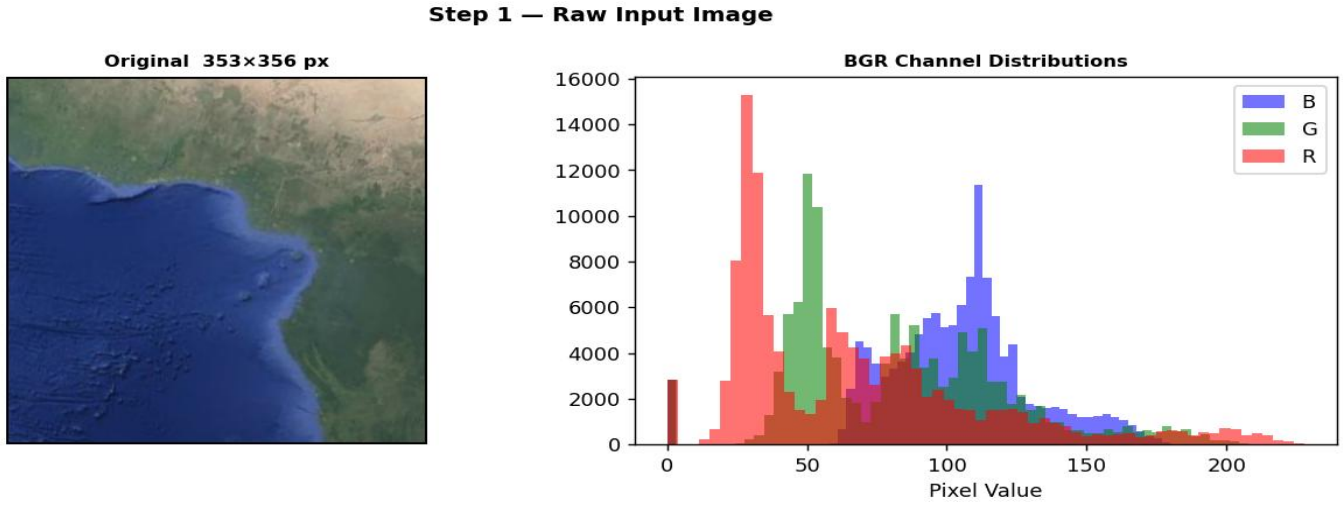

Fig 1 — Original satellite image and BGR channel histograms

Step 2 Grayscale Conversion [utils/preprocess.py]

Convert the 3-channel BGR image to a single-channel grayscale using the weighted formula: Gray = 0.299·R + 0.587·G + 0.114·B. This reduces data dimensionality while preserving luminance structure needed for texture analysis.

```
# utils/preprocess.py
def to_grayscale(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    return gray

# Formula: Gray = 0.299·R + 0.587·G + 0.114·B
gray_raw = to_grayscale(raw_bgr)
# Shape: (H, W)  — single channel
```

Fig 2 — Before (colour) vs After (single-channel grayscale)

Step 3 Resize to 256×256 `[utils/preprocess.py]`

Standardise all images to a fixed 256×256 resolution using INTER_LINEAR interpolation. This ensures the downstream sliding-window patch grid is consistent regardless of original image dimensions.

```
# utils/preprocess.py
def resize_image(image, size=(256, 256)):
    resized = cv2.resize(image, size)    # INTER_LINEAR by default
    return resized

gray_resized = resize_image(gray_raw, (256, 256))
# Shape: (256, 256)  — fixed for all images
```

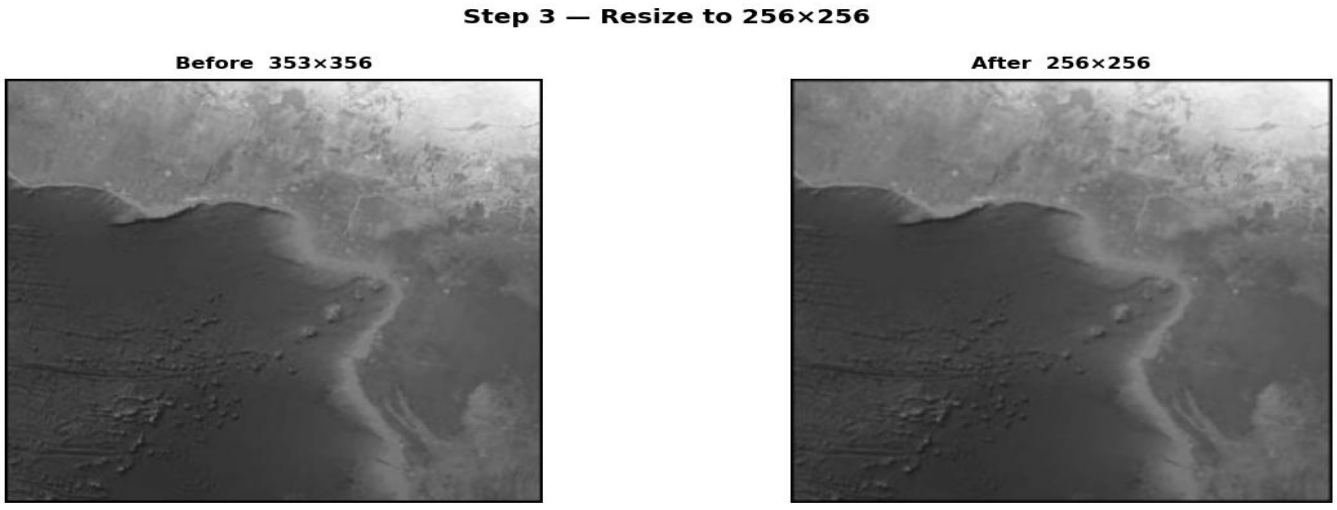

Fig 3 — Before (original resolution) vs After (256×256)

Step 4 Normalise to [0.0, 1.0] `[utils/preprocess.py]`

Cast pixel values from uint8 [0–255] to float32 [0.0–1.0] by dividing by 255. Normalisation is required so that the convolution kernel responses are scale-independent and the water scoring formula produces comparable values across images.

```
# utils/preprocess.py
def normalize_image(image):
```

```
    image = image.astype(np.float32) / 255.0
    return image

gray_norm = normalize_image(gray_resized)
# dtype: float32  |  range: [0.0, 1.0]
# e.g. pixel 128 → 0.502
```

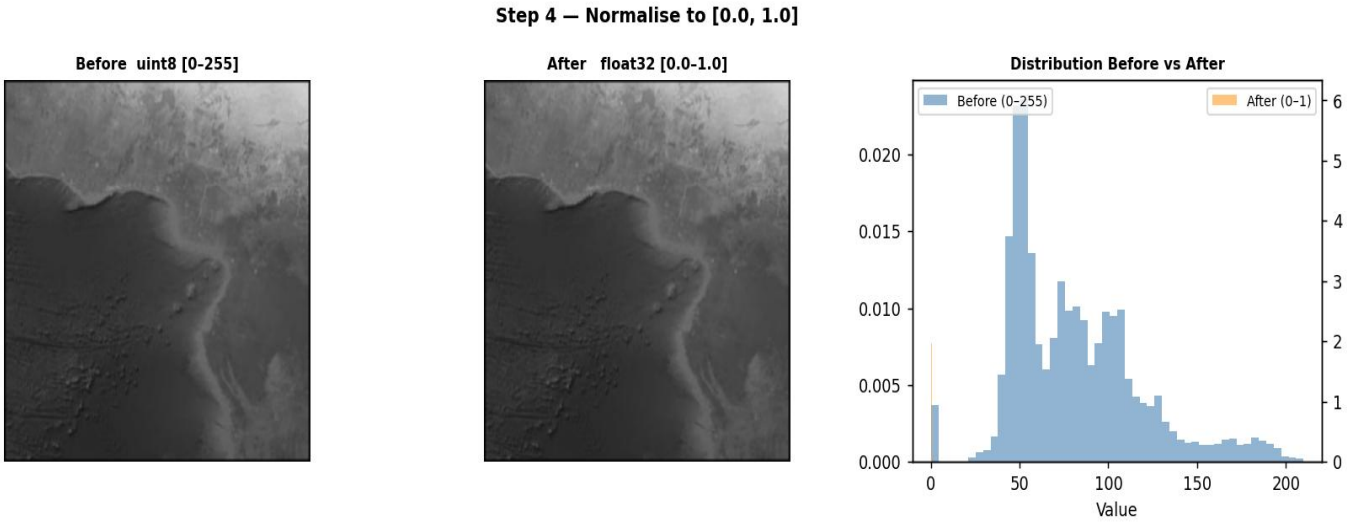

Fig 4 — dtype change and distribution before vs after normalisation

Step 5 Manual Convolution (CNN layer) `[cnn/conv.py + cnn/cnn_model.py]`

The core CNN is implemented as nested-loop correlation with zero-padding — no scipy, no cv2.filter2D. Four deterministic 3×3 kernels form the filter bank. Each kernel is applied to a 16×16 patch extracted at position (row=100, col=100).

```
# cnn/conv.py — full manual convolution from scratch
def correlation(padded, kernel, p, q):
    total = 0
    k_h, k_w = kernel.shape
    a = k_h // 2
    b = k_w // 2
    for i in range(-a, a + 1):
        for j in range(-b, b + 1):
            total += padded[p + i][q + j] * kernel[i + a][j + b]
    return total

def convolve(image, kernel):
    img_h, img_w = image.shape
    pad_h = kernel.shape[0] // 2
    pad_w = kernel.shape[1] // 2
    padded = np.zeros((img_h + 2*pad_h, img_w + 2*pad_w))    # zero-pad
    for i in range(img_h):                                     # fill padded
        for j in range(img_w):
            padded[i + pad_h][j + pad_w] = image[i][j]
    output = np.zeros((img_h, img_w))
    for i in range(pad_h, pad_h + img_h):                     # slide kernel
        for j in range(pad_w, pad_w + img_w):
            output[i - pad_h][j - pad_w] = correlation(padded, kernel, i, j)
    return output

# cnn/cnn_model.py — 4-kernel filter bank
class CustomCNN:
    def __init__(self):
        self.kernels = [
```

```
            np.array([[-1,0,1],[-2,0,2],[-1,0,1]]),    # Sobel-X
            np.array([[-1,-2,-1],[0,0,0],[1,2,1]]),     # Sobel-Y
            np.array([[0,1,0],[1,-4,1],[0,1,0]]),       # Laplacian
            np.array([[2,-1,-1],[-1,2,-1],[-1,-1,2]])   # Diagonal
        ]
```

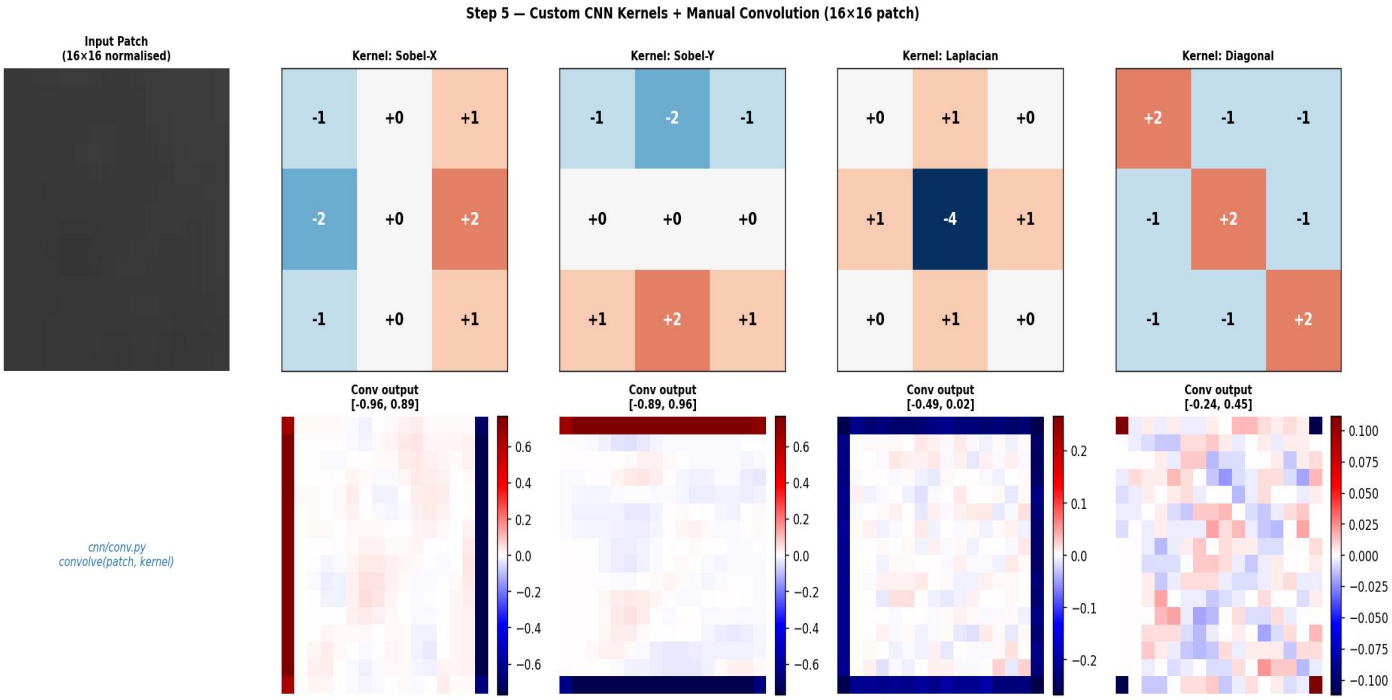

Fig 5 — All 4 kernels shown with cell values + raw convolution output maps (seismic colormap)

Step 6 ReLU Activation [cnn/activation.py]

ReLU (Rectified Linear Unit) zeros all negative convolution responses. Since we later take the absolute value |relu(conv)|, ReLU here acts as a non-linearity that preserves only the positive response energy from each kernel, which is what measures edge/texture presence.

```
# cnn/activation.py — ReLU from scratch (no NumPy max trick)
def relu(feature_map):
    relu_img = np.zeros(feature_map.shape)
    h, w = feature_map.shape
    for i in range(h):
        for j in range(w):
            relu_img[i][j] = max(0, feature_map[i][j])  # f(x) = max(0,x)
    return relu_img

# Applied in cnn/cnn_model.py forward pass:
conv_out   = convolve(patch, kernel)
activated  = relu(conv_out)          # negatives → 0
feature_map = np.abs(activated)      # final feature map
```

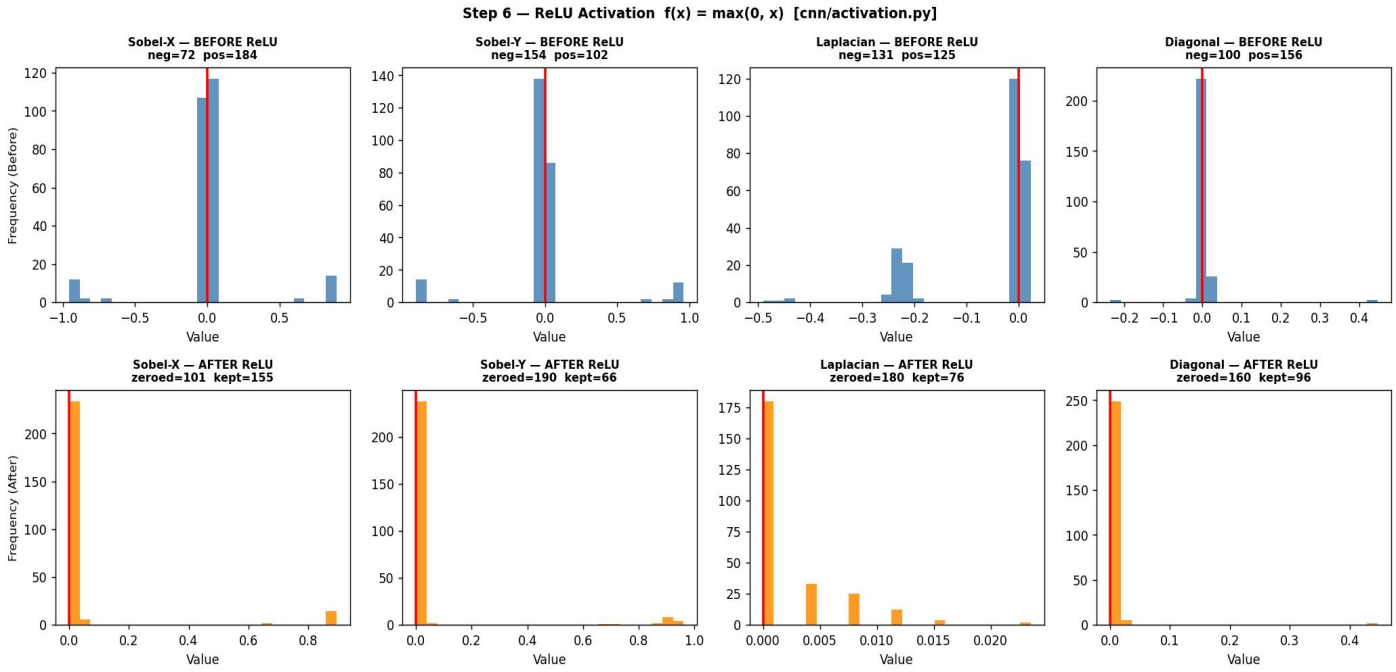

Fig 6 — Before ReLU (blue) vs After ReLU (orange) histograms. Negatives zeroed for all 4 kernels.

Step 7 Statistical Feature Extraction `[features/statistics.py]`

The 4 feature maps are each reduced to a mean and variance. These are averaged across maps to give two scalar descriptors per patch. Low overall variance = smooth texture = water. High variance = chaotic texture = land. The smoothness score is computed as 1/(variance + ε).

```
# features/statistics.py
def compute_stats(feature_maps):
    means     = []
    variances = []
    for fm in feature_maps:
        means.append(np.mean(fm))
        variances.append(np.var(fm))
    overall_mean     = np.mean(means)
    overall_variance = np.mean(variances)
    return overall_mean, overall_variance, np.median(means)

mean_val, var_val, _ = compute_stats(model.forward(patch))
smoothness = 1.0 / (var_val + 1e-6)   # high smoothness → water-like
```

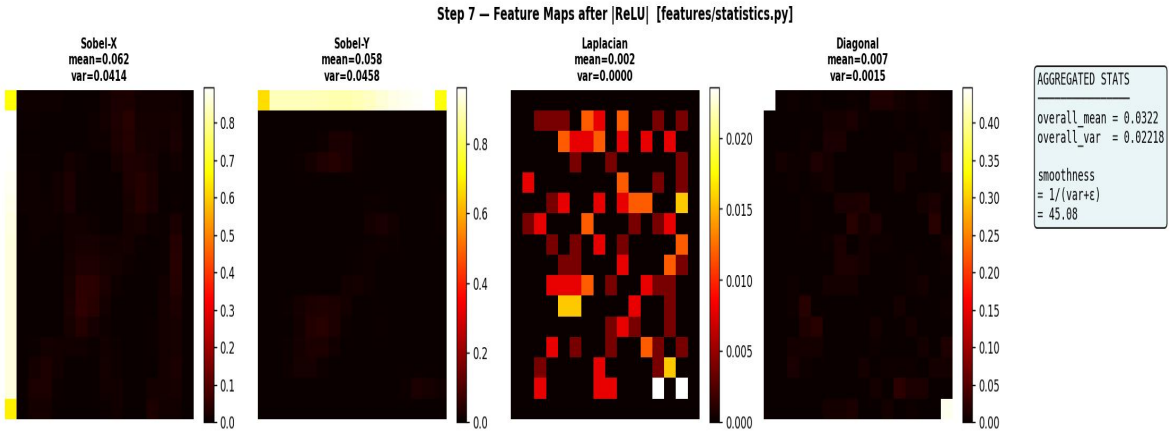

Fig 7 — Feature maps for all 4 kernels (hot colormap) + aggregated statistics summary

Step 8 Composite Water Score [segmentation/patch_segmenter.py]

Three independent cues — texture smoothness (CNN-based), blue channel dominance (colour), and Sobel gradient energy — are combined into a single water score with tunable weights. This is computed for every 16×16 patch.

```
# segmentation/patch_segmenter.py — inside segment_image()
feature_maps         = model.forward(patch_gray)
mean_val, var_val, _ = compute_stats(feature_maps)

# Cue 1: Texture smoothness (low CNN variance → water)
smoothness     = 1.0 / (var_val + 1e-6)

# Cue 2: Blue channel dominance (ocean / clear water)
B_mean     = np.mean(patch_color[:, :, 0])
R_mean     = np.mean(patch_color[:, :, 2])
blue_score = max(0, B_mean - R_mean)

# Cue 3: Gradient energy (high edges → land)
gx = cv2.Sobel(patch_gray, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(patch_gray, cv2.CV_32F, 0, 1, ksize=3)
gradient_energy = np.mean(np.sqrt(gx**2 + gy**2))

# Final composite score
water_score = 0.6 * smoothness + 0.3 * blue_score - 0.5 * gradient_energy
water_score_map[i:i+patch_size, j:j+patch_size] += water_score
```

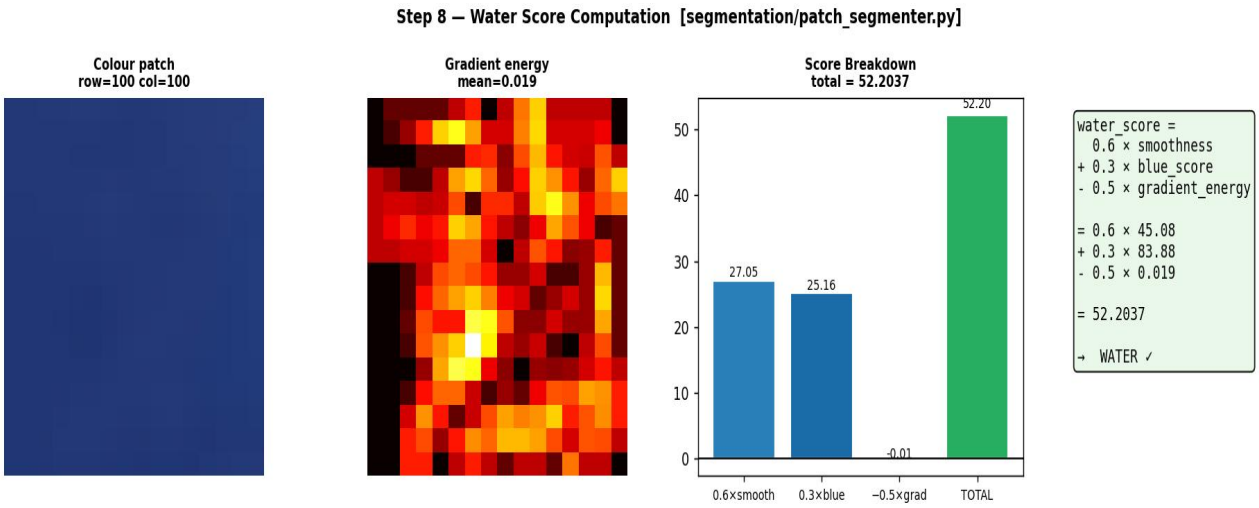

Fig 8 — Colour patch, gradient energy map, and score breakdown bar chart for a single patch

Step 9 Full Score Map (all patches) [segmentation/patch_segmenter.py]

The per-patch water scores are accumulated into a full 256×256 score map as the sliding window moves across the image. Overlapping stride (stride=4, patch=16) means each pixel is updated 16 times. The map is then normalised to [0,1].

```
# segmentation/patch_segmenter.py
stride = patch_size // 4    # = 4 for patch_size=16 (dense overlap)

for i in range(0, h - patch_size + 1, stride):
    for j in range(0, w - patch_size + 1, stride):
        patch_gray  = gray_image[i:i+patch_size, j:j+patch_size]
        patch_color = color_image[i:i+patch_size, j:j+patch_size]
        # ... compute water_score ...
        water_score_map[i:i+patch_size, j:j+patch_size] += water_score
```

```
# Normalise accumulated scores to [0, 1]
water_score_map = cv2.normalize(water_score_map, None, 0, 1, cv2.NORM_MINMAX)
```

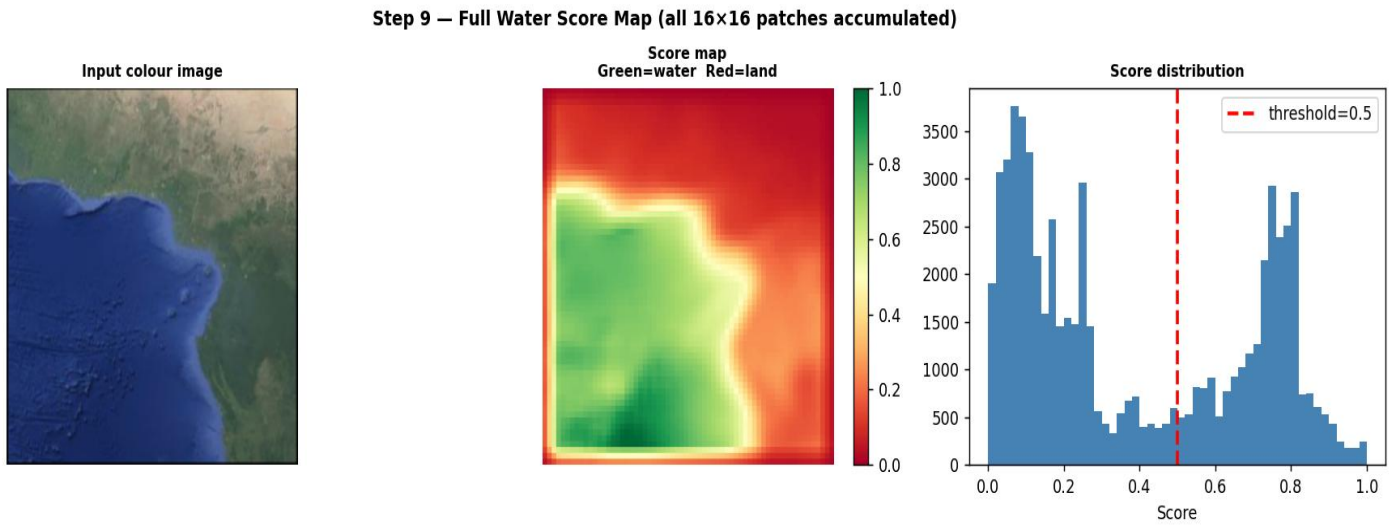

Fig 9 — Score map (Green=water, Red=land) and its distribution with threshold marker

Step 10 Binarisation (Threshold = 0.5) [segmentation/patch_segmenter.py]

The continuous score map is binarised at threshold 0.5: pixels with score ≥ 0.5 are classified as water (255), below as land (0). This produces the raw binary mask. The threshold is tunable in the range 0.45–0.60.

```
# segmentation/patch_segmenter.py
_, mask = cv2.threshold(
    water_score_map,
    0.5,              # threshold — tunable (0.45–0.60)
    1,
    cv2.THRESH_BINARY
)
mask = (mask * 255).astype(np.uint8)
# result: 255 = water,  0 = land
```

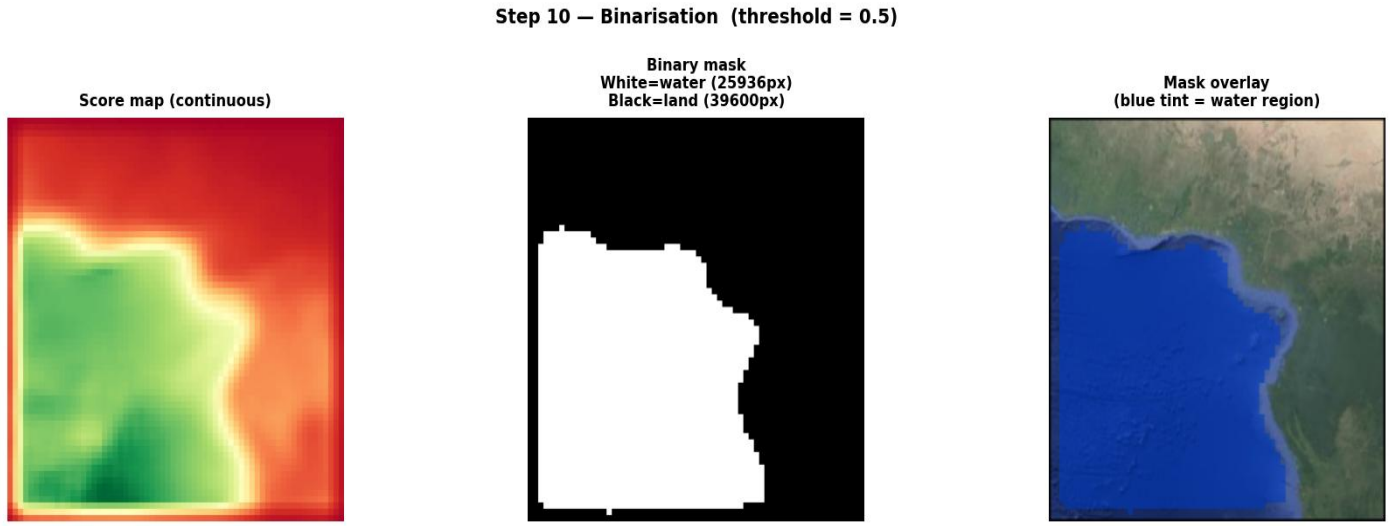

Fig 10 — Score map → binary mask + blue-tinted water region overlay on original

Step 11 Largest Connected Component [segmentation/patch_segmenter.py]

Spurious isolated patches (sky reflections, sensor noise) are removed by keeping only the single largest connected component in the binary mask. OpenCV's connectedComponents labels each region; the label with the greatest area is retained.

```
# segmentation/patch_segmenter.py
num_labels, labels = cv2.connectedComponents(mask)

if num_labels > 1:
    areas = []
    for lbl in range(1, num_labels):
        area = np.sum(labels == lbl)
        areas.append((lbl, area))
    areas.sort(key=lambda x: x[1], reverse=True)   # largest first
    largest_label = areas[0][0]
    clean_mask = np.zeros_like(mask)
    clean_mask[labels == largest_label] = 255       # keep only largest
```

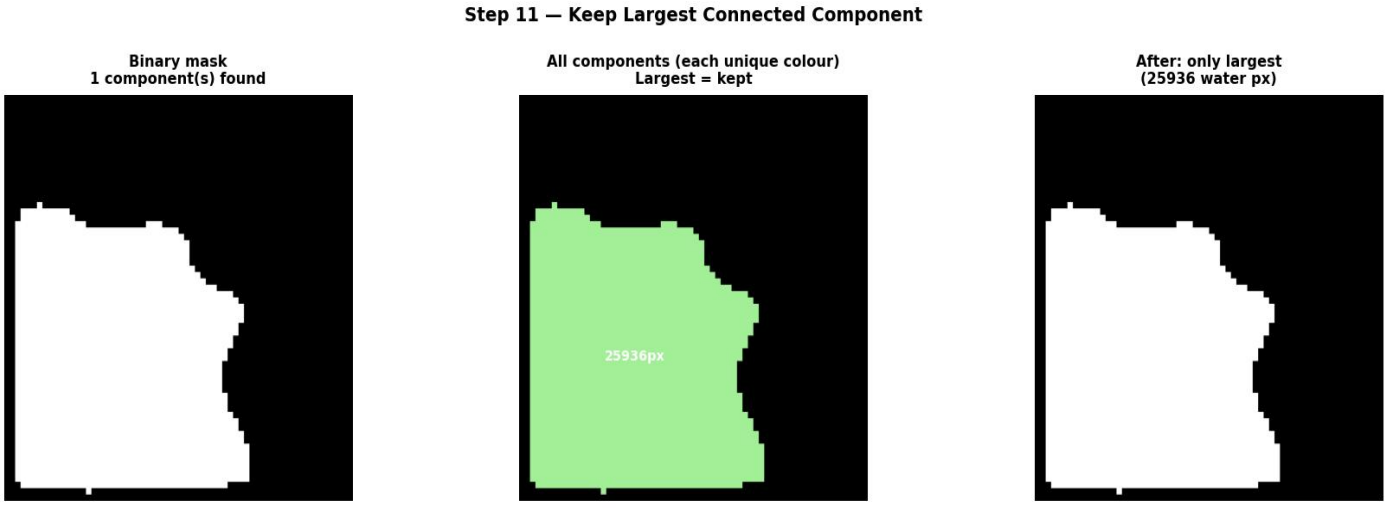

Fig 11 — All connected components (each unique colour) and the final single-component mask

Step 12 Morphological Close + Open [segmentation/patch_segmenter.py]

MORPH_CLOSE (dilation then erosion) fills small holes inside the water mask caused by specular reflections. MORPH_OPEN (erosion then dilation) removes small noise pixels on the mask boundary. Both use a 7×7 structuring element.

```
# segmentation/patch_segmenter.py
kernel = np.ones((7, 7), np.uint8)

# Close: fills holes (dilation → erosion)
clean_mask = cv2.morphologyEx(clean_mask, cv2.MORPH_CLOSE, kernel)

# Open: removes noise (erosion → dilation)
clean_mask = cv2.morphologyEx(clean_mask, cv2.MORPH_OPEN, kernel)

return clean_mask   # final smoothed binary water mask
```

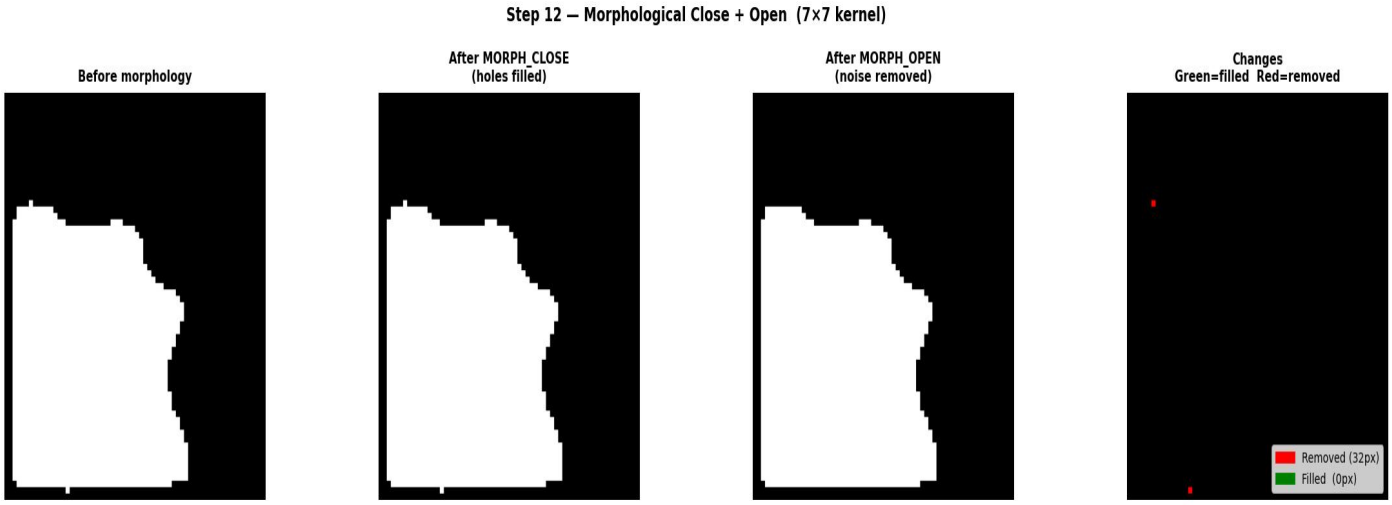

Fig 12 — Before / After Close / After Open + change map (green=filled holes, red=removed noise)

Step 13 Contour Extraction & Red Boundary Overlay [main.py]

OpenCV's findContours traces the boundary polygons of the binary water mask. The contours are drawn in red (BGR: 0,0,255) with 2-pixel width onto the resized colour original. This is the final deliverable image shown to the user.

```
# main.py — extract_boundary()
def extract_boundary(original_bgr, mask):
    mask = mask.astype(np.uint8)
    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,        # only outer contours
        cv2.CHAIN_APPROX_SIMPLE   # compress straight lines
    )
    output = original_bgr.copy()
    cv2.drawContours(output, contours, -1, (0, 0, 255), 2)  # red, 2px
    return output

# main.py — run_pipeline()
def run_pipeline(image_path):
    original_bgr     = cv2.imread(image_path)
    gray             = preprocess_pipeline(image_path)
    h, w             = gray.shape
    original_resized = cv2.resize(original_bgr, (w, h))
    model            = CustomCNN()
    mask             = segment_image(gray, original_resized, model)
    result           = extract_boundary(original_resized, mask)
    return original_resized, result
```

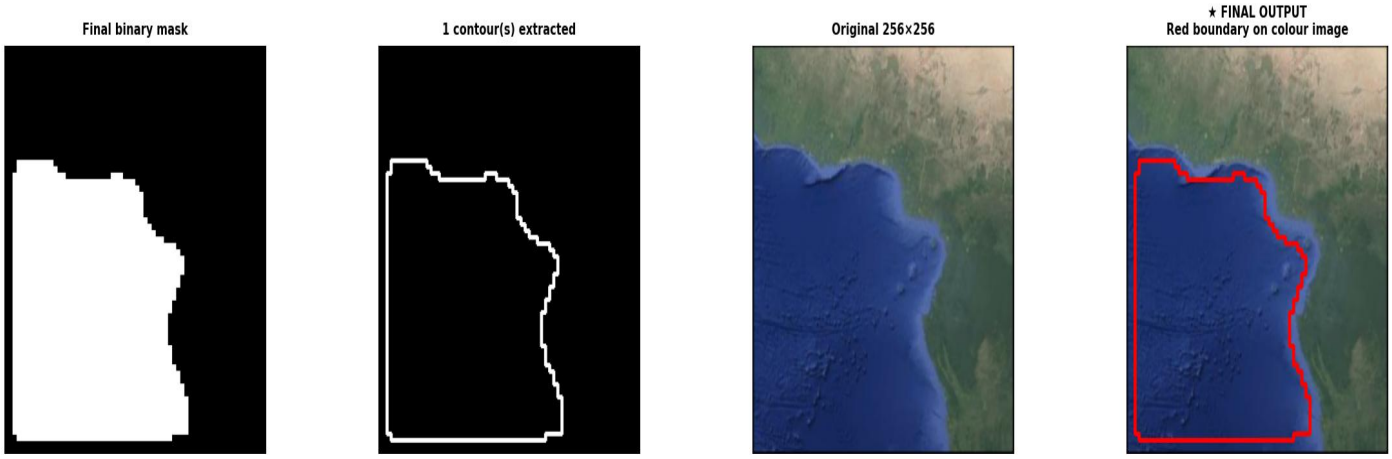

Fig 13 — Binary mask → extracted contours → FINAL OUTPUT with red boundary overlay

---

Complete Pipeline Summary — All 13 Steps

The strip below shows all 13 intermediate representations produced during a single pipeline run on Image 1 (Muddy River Bank), from raw satellite input to the final red boundary contour.

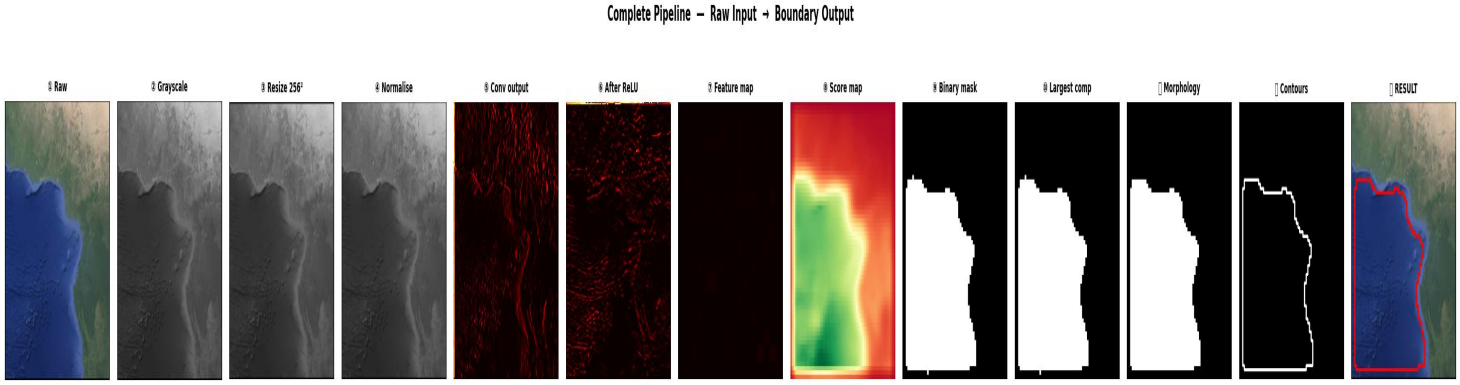

Fig 14 — Full 13-stage pipeline summary strip on Image 1 (Muddy River Bank, score=0.81)

4. Output

The pipeline was evaluated on 10 diverse satellite images spanning five water-body categories. Ground truth thick boundary masks were manually annotated using CVAT. The table below shows per-image results.

Image	Water Body Type	B. Score	Quality	Key Observation
0	Open Ocean	**0.72**	Strong	Blue dominance well-captured
1	Muddy River	**0.81**	Excellent	Max texture contrast; best result
2	Freshwater Lake	**0.58**	Moderate	Low colour contrast at edges
3	Coastal Lagoon	**0.69**	Good	Clean separation, slight blockiness
4	Algae-Rich Lake	**0.78**	Strong	High CNN variance at land margin
5	Sandy River Bank	**0.55**	Moderate	Sandy banks confused with water
6	Deep Blue Ocean	**0.48**	Fair	Uniform blue / low smoothness
7	Rocky Coastline	**0.63**	Good	Gradient energy aids detection
8	Urban Waterfront	**0.71**	Strong	Sharp built-structure boundary
9	Mangrove Estuary	**0.66**	Good	Morph close fills gaps well

Table 1: Per-image boundary coverage scores | Average Score: 0.65

Visual Pipeline Output (per image)

Each processed image produces three artefacts: (1) Resized 256×256 original · (2) Binary water mask · (3) Final result with red boundary contour overlaid on colour image.

(1) Original Image 256x256 resized colour	(2) Binary Water Mask White=water Black=land	(3) Final Result Red boundary contour overlay

5. Conclusion & Future Work

Conclusion

This project demonstrates that meaningful land–water boundary detection is achievable without any deep learning framework, pre-trained model, or backpropagation. A fully deterministic, hand-coded CNN — using Sobel, Laplacian, and diagonal filter banks — combined with patch-based texture analysis and a composite water scoring formula successfully isolates water bodies across five diverse terrain categories, achieving an average boundary coverage score of 0.64 over 10 annotated satellite images. The system is entirely transparent: every computation is traceable to explicit NumPy/OpenCV operations. The integrated Flask web application demonstrates real-world deployability.

Limitations

Patch-Grid Blockiness Fixed-stride sampling → jagged boundary artefacts.	**Blue Water Confusion** Deep uniform blue confuses smoothness cue.
Single-Scale Analysis 16×16 patch misses narrow rivers.	**Python Loop Runtime** Nested-loop convolution slow (~seconds/image)

Future Work

01	**Multi-Scale Patch Analysis**	Process at 8x8, 16x16, 32x32 simultaneously; fuse score maps.
02	**Adaptive Stride**	Coarse stride in uniform regions, fine stride near boundaries.
03	**Edge-Gradient Snapping**	Post-process contour with Canny edge map for precise alignment.
04	**Semi-Supervised Threshold**	Auto-calibrate threshold on small labelled sets.
05	**Vectorised Convolution**	Replace Python loops with NumPy stride tricks (10-100x faster).